

# Reducing Flakiness in End-to-End Test Suites: An Experience Report

Dario Olianas[1(✉)], Maurizio Leotta[1], Filippo Ricca[1], and Luca Villa[2]

[1] Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi (DIBRIS), Università di Genova, Genova, Italy
dario.olianas@dibris.unige.it, {maurizio.leotta,filippo.ricca}@unige.it

[2] IRCCS Istituto Giannina Gaslini, Clinica Pediatrica e Reumatologia, PRINTO, Genoa, Italy
lucavilla@gaslini.org

**Abstract.** End-to-end (E2E) testing, a technique employed to assure quality of web applications, is cost-effective only if the test suite is not flaky. Flaky test scripts produce non-deterministic results that undermine testers' trust and thus the usefulness of the entire test suite.

Recently, we were involved in the refactoring of an existing automated flaky E2E test suite for a large Web application. In this paper, we report on our experience. During the refactoring, we have computed the effort made and formalized the procedure we followed in algorithmic way so that our experience can also be of help to other developers/testers. Our procedure allowed to reduce the flakiness to virtually zero w.r.t. the original flaky test suite. Moreover, as a positive side effect, the execution time of the test suite has been reduced by of the 57%.

**Keywords:** E2E testing · Flakiness · Empirical study

## 1 Introduction

Recently, many companies are gearing up to have automated test suites capable of quickly detecting bugs during software evolution. In fact, if implemented correctly, the automated test suites are able to bring many benefits to companies, such as better quality software and cost reduction [6,16]. However, these benefits can only be achieved if the automated test suite is reliable and has no flakiness problems.

A test script is *flaky* when may non-deterministically pass or fail on the same version of the Application Under Test (AUT), i.e. leading to different results in different runs on the same AUT without any change in both the app and test code [5,22]. The flakiness problem is very insidious for companies because: (1) it makes lose confidence in the results of the execution of the test suites with false alarms, (2) it increases deployment/release times, and generally, (3) it increases development costs [22]. Many big companies, such as Google, Facebook and Microsoft are facing this problem [22] and unfortunately effective solutions that

© Springer Nature Switzerland AG 2021
A. C. R. Paiva et al. (Eds.): QUATIC 2021, CCIS 1439, pp. 3–17, 2021.
https://doi.org/10.1007/978-3-030-85347-1_1

allow to reveal and resolve the flakiness do not exist yet. Most of the proposed methods depend on test repetition, i.e. a test script is applied to the same AUT for a given number of times, if the results are different, then the test is marked as flaky [13].

Recently the authors were involved in the refactoring of an existing automated End-to-End (E2E) test suite concerning a large multi-page Web application for paediatric rheumatology patients management. The partner organization is PRINTO, a not for profit, non governmental, international research medical network with the goal to foster, facilitate and coordinate the development of clinical studies in children with paediatric rheumatic diseases. The testsuite was developed in Java language using the Selenium WebDriver [19] and TestNG [2] frameworks and had relevant flakiness problems that created continuous false alarms.

To face and limit the flakiness problem, we considered tools and best practices available in the state-of-the-art literature, utilizing also the experience gained from our past scientific [7,10,12,21] and industrial [8,9] collaborations in developing E2E test suites for web applications. After a thorough analysis of the problem, we used a well-defined manual procedure based mainly on substitution rules of thread-sleep commands located in test scripts with a better alternative, WebDriverWait commands, based on a polling mechanism built into Selenium able to wait for an element to appear or exist, and to wait for some sort of change in a web page. We have quantified the effort made and formalized the procedure in algorithmic way so that our experience can also be of help to other developers/testers.

This paper is organized as follows: Sect. 2 describes the PRINTO web app and the test suite we refactored. Section 3 sketches the approach we used to limit the flakiness problem present in the original test suite. Section 4 reports the data on the reduction of flakiness compared to the original test suite. Section 5 presents lessons learnt and gives future extensions. Finally, Sect. 6 summarizes related works and Sect. 7 concludes the paper.

## 2 The PRINTO Web App and the Associated E2E Test Suite

### 2.1 The PRINTO Web Application

The Paediatric Rheumatology INternational Trials Organisation (PRINTO) is an academic international research network with the goal to facilitate and coordinate multi-centre international clinical trials and registries in children with paediatric rheumatic diseases and autoinflammatory diseases. Having the need to collect clinical information worldwide, PRINTO has developed and maintained during the years a large multi-page web application (www.printo.it) to achieve its goal. The PRINTO web application is used by more than 500 centers worldwide with about 1500 members today and it provides several functionalities to collect information for clinical trials, disease registries, safety registries and clinical consensus processes. It was developed on the server side in PHP language (approximately 100k LOCs[1]) and on the client side in HTML and JavaScript (for the latter, approximately 2k LOCs).

[1] Computed using Locmetrics (https://www.cheonghyun.com/blog/120).

### 2.2 Original PRINTO's E2E Test Suite

The original E2E test suite we refactored is a Java test suite that relies on Selenium WebDriver [19] to interact with the application's pages. The test suite has been built with Maven, and it uses the testing framework TestNG [2]. It uses the GeckoDriver implementation for the browser Mozilla Firefox and is designed following the Page Object model [1], i.e., a model where Object Oriented classes serves as interfaces to Web pages. It contains 11707 LOCs, namely 5523 in the page objects and 6184 in the test scripts. It is composed by 169 independent test scripts that can be executed in any order. The independence of test scripts has been obtained by the accurate use of `Setup` and `Teardown` TestNG methods, respectively, methods that are executed before a test script (or a test class) to set up the initial state, and after a test script (or test class) to undo the test script actions with the goal of resetting the initial state of the application. The original test suite contains 82 of such methods, also known as *test fixtures*, for a total of 251 methods (test scripts plus test fixtures). Both test scripts and test fixtures can be executed multiple times in a single run of the test suite. The test fixtures are executed several times according to the specific TestNG method (for example, a `BeforeMethod` method is executed for each test script it belongs to its test class) while test scripts, that are in several cases parameterized, are executed multiple times, depending on the number of different values passed to them. In total, 551 methods are executed for each execution of the testuite (of which 334 test fixtures).

**Statistics on the Test Suite Execution.** The original test suite was heavily affected by the flakiness problem. To quantify the problem, we executed the original test suite in a controlled environment (i.e., without modifying the test suite and system under test during runs) for 100 times and obtained that, on 55100 total method executions (counting both test fixtures and test scripts), 54699 (**99.27%**) passed, 44 (**0.08%** failed) and 357 (**0.65%**) were skipped. In TestNG, a test script is skipped when its corresponding `Before` method fails: because of that, flakiness is still more problematic when it happens in a `Before` method, since it will prevent other test scripts to be executed. The above reported number of false alarms may seem negligible with respect to the total number of test methods executions, but if we count every execution of the test suite in which at least one method failed, we have 18 failures out of 100 executions, giving a flakiness score for the test suite of **18%**. Since the original test suite runs every night, approximately means a false failure report every 5 days. Looking at the finer grained results, we realized that flakiness is quite localized: out of 251 test methods, only 26 failed at least one time during the 100 executions. For these methods, we have computed the flakiness scores as: number of times the method failed divided by number of times the method passed. The maximum flakiness score was 3.09% for two methods (i.e., the most flaky). Finally, concerning execution times, the original test suite required on average 81 min to run (value calculated by averaging the 82 runs that were not affected by flakiness over a total of 100).

Copyrighted material

## 3 Overview of the Approach for Reducing Flakiness

As a first step to discover the reasons for the flakiness of the test suite, we have analyzed in detail the code of the test scripts and the page objects that compose it. The first thing we noticed was the presence of `thread sleeps` positioned both in the code of the page objects and in that of the test scripts. `Thread sleeps`, i.e., function calls that pause the execution of a test script for a given time, are usually employed by developers for managing *asynchronous calls*. In the context of web applications, asynchronous calls happen whenever there is an interaction with a page that requires some data to be retrieved from the web server. However, this solution has two disadvantages, indeed thread sleeps: a) are inefficient, especially in large test suites, because they always wait the same amount of time, defined a priori by the developer, even when the page finishes to load more quickly and, b) are often one of the reasons of flakiness, as testified for example in Luo et al. [13] and Ricca and Stocco papers [17]. The problem is that thread sleeps often handle asynchronous calls 'badly', without properly waiting for the web server response, thus leading to flakiness. From several years now the Selenium testing framework has provided a smarter mechanism for managing asynchronous calls, called `explicit wait`. Technically speaking, an `explicit wait` is a Java object of class `WebDriverWait`, that can be used in combination with an `ExpectedCondition`, that is a function that informs the explicit wait about which condition must be verified to stop waiting. From the Selenium documentation[2] we can identify six main categories of expected conditions:

- expected conditions checking the visibility of an element
- expected conditions checking the clickability of an element
- expected conditions checking the presence of an element
- expected conditions checking the number of elements
- expected conditions checking the page's URL
- expected conditions for text comparison
- expected conditions checking DOM attributes

Explicit waits are more efficient w.r.t. thread sleeps because they stop waiting when the element in question is ready for the specific interaction, and if we consider all the test scripts in a large test suite the usage of explicit waits can lead to great time savings. Moreover, explicit waits are more flexible and more reliable, because they allow to check for complex conditions to be verified: for example, if we have a text in a page that can be dynamically updated via AJAX, it would be difficult to check it using thread sleeps, but with the `textToBe` expected condition offered by explicit waits this control becomes feasible.

As already mentioned, the original test suite was implemented using thread sleeps. After a first analysis, we realized that they were set incorrectly: some

[2] Documentation for the ExpectedConditions class https://www.selenium.dev/selenium/docs/api/java/org/openqa/selenium/support/ui/ExpectedConditions.html.

were missing in some points and some had too short or too long waiting times. The presence of these thread sleeps could be one of the reasons for the flakiness and also could explain the very long execution times with respect to the number and length of executed test scripts. So, we decided to modify the existing test suite with a two-steps approach. First, by adding or modifying the thread sleeps in all the points where failures occurred. As a result, we were able to reduce flakiness, but at a cost of an increased total execution time: from 81 min to 128 min. Second, we decided to refactor this time-consuming version of the test suite by changing every thread sleep (both those already present in the original test suite and those introduced by us to limit flakiness) with explicit wait, relying on a phased approach to make sure we do not introduce novel regressions (i.e. some tests that previously worked fine start to fail or to be flaky) during the refactoring.

### 3.1 Test Suite Refactoring

In this subsection, we will describe the refactoring procedure we used for PRINTO test suite. Since the flakiness problem is, by definition, non-deterministic, the procedure is based on heuristics and follows a step-by-step approach to detect possible regressions as soon as possible (i.e., immediately after their introduction). In our case, we have a regression when a refactored test script that previously passed starts to fail, also in non-deterministic way; in this last case, it means that our refactoring worsened flakiness rather than improving it. The procedure **`RemoveSleep`** is described in Algorithm 1, and it can be summarized as follows: for each thread sleep in the test suite, we replace it with the appropriate explicit wait (**`NewExplicitWait`**), then, we select a test script that calls the method containing the introduced explicit wait (most likely a page object method, since thread sleeps have been mainly inserted in page object methods) and execute it $K = 3$ times (**`Validate`**). If it passes $K$ times, we consider the next thread sleep, otherwise we try to fix the test script using different strategies (**`FixExplicitWait`**). In the worst case, i.e., if all fixing attempts fail, we give up and restore the original thread sleep: in our case, this happened only four times on 196 thread sleeps. The choice of using a single test script to validate an explicit wait is mainly motivated by time-saving requirements. A completely safe solution would be to run every test script calling the PO method where the introduced explicit wait is used, but this would enormously increase the required time. So, to compensate for the adopted simplification, every $W = 15$ thread sleeps replaced, we execute the whole test suite $H = 3$ times to check that the inserted explicit waits do not introduce novel regressions: this check can help us to find previously undetected regressions. When this happen, we troubleshoot the regression by applying **`FixExplicitWait`** to the points of failure. The selection of 15 ($W$) as the number of changes before re-executing the entire test suite for three times ($H$) is part of our heuristics. We selected that number since it appears to be a reasonable trade off between (a) the number of modifications: increasing this number can make difficult to understand the cause of a possible regression and (b) the time required to execute the test suite: executing it too often could slow down too much the refactoring process.

needed to employ a different one, i.e., *presenceOfElementLocated*. In total, we had to apply the **`FixExplicitWait`** sub-procedure 10 times: we changed the expected condition with *presenceOfElementLocated* four times; we added another explicit wait in a different point of the page objects' code two times; we removed the explicit wait and restored the thread sleep four times. The average time it took to replace these thread sleeps was much longer than the others (20.3 min vs. 8.9 min).

## 4 Quantifying the Obtained Improvements

In this section, we describe the improvements of the refactored test suite with respect to the previous versions. Before applying the procedure to replace thread sleeps with explicit waits, we tried to reduce flakiness by adding thread sleeps in points where failures were more frequent. After applying the procedure to replace thread sleeps with explicit waits, we ran the test suites 100 times to quantify the obtained improvements, both on flakiness reduction and on execution time reduction. We executed the test suites on a Debian GNU/Linux 10 virtual machine with 6 GB of RAM assigned, hosted on a Windows 10 Pro laptop with 16 GB of RAM and Intel Core i3 10110U CPU with four logical cores (two physical). To avoid interferences, no other applications were running on the host system and on the virtual machine. The test suite has been executed using Jenkins[3], a popular continuous integration tool used by PRINTO to run the test suite. The system under test, instead, is deployed on a cloud solution. We decided to provide results on flakiness reduction at two levels of granularity, respectively at test method granularity and at test suite execution granularity.

### 4.1 Test Method Granularity Analysis

In Table 2, the column *Total test methods executions* reports the total number of executed test methods obtained running the three test suites 100 times, including test fixtures and multiple executions of the same test script with different

**Table 2.** Validation results at test method granularity

| | Test methods executions | Passed | Failed | Skipped |
|---|---|---|---|---|
| Original test suite | 55100 | 54699 *(99.270%)* | 44 *(0.080%)* | 357 *(0.650%)* |
| Refactored test suite with thread sleep | 55100 | 55099 *(99.998%)* | 1 *(0.002%)* | 0 *(0.000%)* |
| Refactored test suite with explicit waits | 55100 | 55092 *(99.990%)* | 8 *(0.010%)* | 0 *(0.000%)* |

[3] Jenkins website https://www.jenkins.io/.

parameters. The other columns report respectively: number and percentage of methods passed, failed and skipped.

For the original version of the test suite, on 55100 total method executions 44 failed and 357 were skipped. The high number of skipped test is due to the fact that some of the flaky test methods were setup methods, executed before other test methods of their classes: as a consequence, when a setup method fails, the whole test class is skipped.

In the thread sleep-based refactored version of the test suite, we can see that only one test script failed on 100 executions. This is the best result we obtained from a flakiness-reduction point of view, but on the other hand the execution time increased from an average of 81 min to an average of 128 min.

In the final version of the test suite (the one with explicit waits), we can see that, on 55100 test methods executions, eight failed against only one for the refactored version with thread sleeps (in both cases we have no test scripts skipped). This would seem to suggest that the version with all the thread sleeps is more reliable, but, however, it is important to note that the eight test scripts that failed in the last version of the test suite happened during the same execution (the ninth execution in time order). Since these failures never happened again in the subsequent 91 executions, we assume that they were not caused by defects of the test suite, but probably by network problems (or other server-side problems).

### 4.2 Test Suite Granularity Analysis

Table 3 reports the flakiness results at test suite execution granularity. The column *Total test suite executions* reports the number of times each test suite was executed, the column *Flaky executions* reports the number of executions in which at least one test method failed and the column *Flakiness score* reports the percentage of flaky executions.

**Table 3.** Validation results at whole test suite execution granularity

| | Total test suite execution | Flaky test suite execution | Flakiness score |
|---|---|---|---|
| Original test suite | 100 | 18 | 18% |
| Refactored test suite with thread sleep | 100 | 1 | 1% |
| Refactored test suite with explicit waits | 100 | 1 | 1% |

This table shows results more interesting from a tester's perspective since reports the probability of having a false alarm after executing the test suite. We can note that the scores of the full thread sleep version and of the explicit wait version are comparable.

### 4.3 Execution Time

Considering the execution time of the three test suites, we can note that the explicit wait based test suite is much faster. In fact, for the explicit wait version we obtained an average execution time of the whole test suite of 35 min, reducing the execution time of 57% with respect to the original, flaky test suite (that required 81 min) and of 72% with respect to the second version of the test suite, with all the required thread sleeps, that required 128 min.

## 5 Discussions, Lessons Learnt, and Future Extensions

In this section, we discuss the obtained results, lessons learnt and possible future extensions of this work.

### 5.1 Following a Precise Procedure Is Extremely Important

We learnt that removing flakiness in a E2E test suite is a hard task. Indeed, the non-deterministic behavior of test scripts makes difficult to understand if a change in the code has positive effects or if, on the contrary, the flakiness is getting worse. During this experience we understood that it is not possible to deal with the flakiness problem without following a precise procedure. For this reason, we defined the procedure described in Algorithm 1. We believe that a well defined procedure helps to minimize subjectivity during refactoring in order to guide programmers with little experience of Selenium WebDriver. Moreover, we learnt that it is fundamental make few changes at a time and rerun the test suite very often. This is because if flakiness occurs after a long series of changes, finding the cause is extremely difficult.

### 5.2 Conservative Solutions Are Too Time-Consuming

A specificity of our procedure is that we use a single test script to validate newly inserted explicit waits. Usually, if the test suite is designed following the page object pattern, a page object method will be used by more than one test script, and so will be any explicit wait inside the method. To be completely sure to not introduce regressions at each insertion of an explicit wait, we should execute multiple times every test script who uses it. However, we learnt that this is prohibitive from the point of view of execution time, so we opted for a time-saving heuristics: running a single test script at each change and then, to find possible undetected regressions, running the whole test suite after a predetermined number of changes.

### 5.3 It Is Hard to Be Too Prescriptive

We learnt that a procedure is useful but it has to be flexible. Indeed, our procedure leaves some degrees of freedom to the developer when it comes to

select expected conditions. In sub-procedure **`NewExplicitWait`**, the developer is asked to choose "an expected condition that checks if the element is visible" or "an expected condition that checks if the element is clickable", but the precise choice of the expected condition is left to the developer. Also in sub-procedure **`FixExplicitWait`** the developer is just asked to list the $k$ most suitable expected conditions for the WebDriver interaction under analysis. This is due to the fact that the precise expected condition to use strictly depends on the characteristics of both the test suite and the application under test. For example, if the test suite adopts the Page Factory pattern [11], the category of expected conditions that take a WebElement as argument, instead of a locator, should be used.

### 5.4 Thread Sleeps Substitution Can be Automated

This work was very important for understanding that thread sleeps replacement (Algorithm 1) can be automated, at least partially. Perhaps this is the most important lesson we learnt. This insight is supported by the data shown in Sect. 3.3, where it is clear that in many cases the thread sleeps have been replaced using the simpler heuristics and the re-execution of the test suite was not always necessary. It is clear that, sub-procedure **`NewExplicitWait`** (insertion of a new explicit wait) and sub-procedure **`Validate`** (validation of a newly inserted explicit wait) are the easiest procedures to automate. For sub-procedure **`NewExplicitWait`**, we just need to specify an expected condition to be used for the two type of considered web page interactions (i.e., read and write). Sub-procedure **`Validate`** is pretty straightforward too, since the list of tests that use the explicit wait under validation can be easily extracted from the test suite's code by means of static analysis techniques. Eventually, the implemented tool should be able to: refactoring the test scripts (i.e., substituting thread sleeps with explicit waits), execute the modified test scripts and collect the results showing statistics about flakiness. Clearly automating the procedure completely is much more challenging, in particular it is difficult to mechanize sub-procedure **`FixExplicitWait`** which is the part where human intervention is most needed. However, it is important to highlight that even the basic implementation of this tool should be able to automate a relevant portion of the refactorings (i.e., 182 out of 196 thread sleep replacements), significantly reducing the time and cost of the entire work.

### 5.5 Manual Intervention Is Needed

As we have already said, the application of the procedure did not allow us to replace all the thread sleeps. In four cases the procedure left the thread sleeps. This is a consequence of our heuristic which does not require the developer executing the refactoring to inspect the DOM of the web pages but relies mainly on the access done by the test scripts to the web pages. We are aware that this choice can be limiting in certain specific cases, however it works very often because web page inspection, we learnt, is rarely necessary. In retrospect, we

investigated why the procedure didn't work in these four cases. In all cases, the problem was due to page elements dynamically loaded via JavaScript: the thread sleep originally inserted allows to complete the loading of the required content, while the explicit waits did not. In one case, we have that a test script compiles a form and sends it to the server. It inserts in the form wrong values, and checks if the web application detects the problem and replies with an error message. The form includes a field with auto-completion and a client-side script is in charge of checking the correctness of the inserted value. With the original thread sleep (inserted just before interacting with the submission button), the client-side check has enough time to complete its execution and shows the expected error. When we applied the procedure, we replaced the thread sleep with an explicit wait that waits for the submission button to be clickable. In this way the sequence of actions is performed correctly, but the client-side validation is skipped because does not have enough time to check the correctness of the inserted value. Thus, the web application responds with a different error from server side and this makes the test script fail. We manually fixed this explicit wait by changing the waited object to the first element that pops up in the auto-completing field: this gives the client-side check enough time to complete its execution and show the client side error. In cases like these it is very difficult (if not impossible) to automate the refactoring and therefore the manual intervention is necessary.

## 6 Related Work

Since the problem of flakiness is often faced in practice by practitioners, there are many works in the scientific literature that consider it.

Among these works we find some that try to understand the reasons of the flakiness and characterize the main causes by means of surveys with professional developers (e.g., [5,13]) or analyzing open source code (e.g., [13,18]). Our experience confirm the results by Eck et al. [5] and Luo et al. [13] on the high diffusion of the *Async category* as main reason of flakiness problems, that happens when a test script performs asynchronous calls without waiting properly the result. Indeed, in our test suite almost all flakiness problems fell into this category. Other works, on the other hand, confirm our choice to replace thread sleeps with explicit waits to alleviate the problem of flakiness as well as reduce execution times. Among these, we find for example the paper by Ahmad et al. [3] where they came to the conclusion that some test smells (e.g., inserting thread sleeps and having dependent test scripts) are the main reasons for test scripts flakiness. This choice of substituting thread sleeps with explicit waits is instead contradicted by Presler-Marshall et al. [15]. In a work similar to ours, carried out by students on a large electronic health record system, the authors came to the conclusion that thread waits give the lowest flakiness, while explicit waits giving the highest.

Finally, there are also many works with a different goal from ours that propose techniques and tools to detect the root cause of a specific case of flaky test script [14] or others that detect flaky test scripts in a test suite [4] or others that deal

19. SeleniumHQ web browser automation (2021). https://www.selenium.dev/. Accessed 08 Apr 2021
20. Shi, A., Lam, W., Oei, R., Xie, T., Marinov, D.: iFixFlakies: a framework for automatically fixing order-dependent flaky tests. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2019, pp. 545–555. Association for Computing Machinery, New York (2019). https://doi.org/10.1145/3338906.3338925
21. Stocco, A., Leotta, M., Ricca, F., Tonella, P.: APOGEN: automatic page object generator for web testing. Softw. Qual. J. (SQJ) **25**(3), 1007–1039 (2017). https://doi.org/10.1007/s11219-016-9331-9
22. Zolfaghari, B., Parizi, R.M., Srivastava, G., Hailemariam, Y.: Root causing, detecting, and fixing flaky tests: state of the art and future roadmap. Softw.: Pract. Exp. **51**, 851–867 (2020)